

6.4 Reading Numbers

The `getline` function of the previous section reads one line from the user, as a *string*. What if we want to read a number? One straightforward way is to read a string as before, and then immediately convert the string to a number. The standard C library contains a number of functions for doing this. The simplest to use are `atoi()`, which converts a string to an integer, and `atof()`, which converts a string to a floating-point number. (Both of these functions are declared in the header `<stdlib.h>`, so you should `#include` that header at the top of any file using these functions.) You could read an integer from the user like this:

```
#include <stdlib.h>

char line[256];
int n;
printf("Type an integer:\n");
getline(line, 256);
n = atoi(line);
```

Now the variable `n` contains the number typed by the user. (This assumes that the user *did* type a valid number, and that `getline` did not return EOF.)

Reading a floating-point number is similar:

```
#include <stdlib.h>

char line[256];
double x;
printf("Type a floating-point number:\n");
getline(line, 256);
x = atof(line);
```

(`atof` is actually declared as returning type `double`, but you could also use it with a variable of type `float`, because in general, C automatically converts between `float` and `double` as needed.)

Another way of reading in numbers, which you're likely to see in other books on C, involves the `scanf` function, but it has several problems, so we won't discuss it for now. (Superficially, `scanf` seems simple enough, which is why it's often used, especially in textbooks. The trouble is that to perform input reliably using `scanf` is not nearly as easy as it looks, especially when you're not sure what the user is going to type.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)